

Digital Signal Processing with FAUST

Xianglei Wu



BACHELORARBEIT

eingereicht am

Fachhochschul-Bachelorstudiengang

Software Engineering

in Hagenberg

im Mai 2026

Betreuung:

Erik Pitzer, FH-Prof. DI (FH) Dr.

© Copyright 2026 Xianglei Wu

Diese Arbeit wird unter den Bedingungen der Creative Commons Lizenz *Attribution-NonCommercial-NoDerivatives 4.0 International* (CC BY-NC-ND 4.0) veröffentlicht – siehe <https://creativecommons.org/licenses/by-nc-nd/4.0/>.

Erklärung

Ich erkläre eidesstattlich, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen nicht benutzt und die den benutzten Quellen entnommenen Stellen als solche gekennzeichnet habe. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt. Die vorliegende, gedruckte Arbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Hagenberg, am 25. Mai 2026

Xianglei Wu

Inhaltsverzeichnis

Erklärung	iv
Preface	vii
Abstract	viii
Kurzfassung	ix
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Forschungsfrage und Zielsetzung	2
1.3 Aufbau der Arbeit	3
2 Grundlagen	5
2.1 Digitale Audiosignale und blockweise Verarbeitung	5
2.1.1 Abtastung und diskrete Sample-Folgen	5
2.1.2 Audio-Blöcke und Echtzeitdeadline	6
2.2 Elementare DSP-Bausteine	6
2.2.1 Gain-Stufe	6
2.2.2 IIR-Filter und Biquad-Struktur	7
2.2.3 FIR-Filter und Faltung	8
2.3 Frequenzanalyse mit DFT und FFT	9
2.3.1 Zeitbereich, Frequenzbereich und DFT	9
2.3.2 Fast-Fourier-Transformation	10
2.4 Performanzrelevante Implementierungsaspekte	11
2.4.1 Compileroptimierung und automatische Vektorisierung	11
2.4.2 Messgrößen	11
2.5 Zusammenfassung	11
3 Faust: Functional Audio Stream	12
3.1 Einführung	12
3.2 Programmmodell	12
3.3 Syntax: Blockdiagramm-Komposition	13
3.4 Semantik	13
3.5 Compiler-Architektur	13
3.6 Codegenerierung	14

3.6.1	Skalarer Modus	14
3.6.2	Vektor-Modus	15
3.6.3	Parallel-Modus	15
3.6.4	Generierte Code	15
3.7	Backends und Architekturdateien	15
3.8	Performance	15
3.9	Zusammenfassung	16
4	Mathematical Elements	17
5	Using Literature	18
6	Printing the Manuscript	19
7	Closing Remarks	20
A	Technical Details	21
B	Supplementary Materials	22
B.1	PDF Files	22
B.2	Media Files	22
C	Questionnaire	23
D	LaTeX Source Code	24
Quellenverzeichnis		25
	Literatur	25
	Online-Quellen	25

Preface

Abstract

This should be a 1-page (maximum) summary of your work in English.

Kurzfassung

An dieser Stelle steht eine Zusammenfassung der Arbeit, Umfang max. 1 Seite. ...

Kapitel 1

Einleitung

1.1 Motivation und Problemstellung

Die digitale Signalverarbeitung (Digital Signal Processing, DSP) ist ein wichtiges Anwendungsfeld in der Informatik besonderen Wert. Ifeachor und Jervis (2002) nennen unter anderem Anwendungen in der Telekommunikation, Audiotechnik, Sprachverarbeitung und Medizintechnik, die mit hohen Anforderungen an Laufzeiteffizienz, deterministische Latenz und numerische Präzision. DSP-Algorithmen werden traditionell in C oder C++ implementiert, unter anderem wegen des direkten Hardwarezugriffs und der hochoptimierten Codeerzeugung Levine und Skolnick, 1997. Die tatsächliche Performance wird im Benchmark untersucht. Trotz all den Vorteilen liegt an der doch noch sehr zeitintensive, fehleranfällige mathematische Übersetzung bis hin zu manuelle Implementierung von DSP-Algorithmen und das exportieren auf verschiedenen Zielplattformen, wie auf Desktop, verschiedensten Embedded Systemen, Web und Plug-in-Hosts; Daraus kristallisierten sich in den letzten zwei Jahrzehnten domänenspezifische Sprachen (Domain-Specific Languages, DSLs) für die Audio-Signalverarbeitung, welches durch Abstraktion und Code-Generierung die Probleme bewältigen soll. (Vgl. Orlarey et al., 2009, S. 2-6)

Orlarey et al. (2009) beschreiben die domänenspezifische Sprache FAUST (Functional Audio Stream), entwickelt am GRAME in Lyon, als eine funktionale, blockbasierte Programmiersprache, die speziell für Audio-DSP entworfen wurde. Programme werden als algebraische Komposition von Signalverarbeitungsoperatoren formuliert. Der FAUST-Compiler übersetzt die mathematisch knappe Beschreibung eines Signalflussdiagramms in mehrere Zielsprachen, darunter C++, Rust, JavaScript und WebAssembly zu finden im GRAME - Centre National de Création Musicale (2025). Aus einer einzigen Quellbeschreibung lassen sich somit Plug-ins für Digital Audio Workstations, Embedded-Anwendungen auf Mikrocontrollern und Webapplikationen erzeugen. (Vgl. Orlarey et al., 2009, S. 2-6)

Die praktische Leistungsfähigkeit dieses Ansatzes wird von Michon und Smith am Beispiel des FAUST-STK demonstriert. Dabei handelt es sich um eine Sammlung virtueller Musikinstrumente, deren Modelle überwiegend auf Wellenleitersynthese und teilweise auf modaler Synthese beruhen. Die aus dem Synthesis ToolKit und SynthBuilder übernommenen Modelle wurden für die FAUST-Semantik vereinfacht und hinsichtlich ihrer Effizienz optimiert.

Die durchgeführten Vergleiche zeigen außerdem, dass die FAUST-Implementierungen im Quelltext in der Regel kompakter sind als die entsprechenden STK-C++-Implementierungen. Bei den untersuchten Pure-Data-Plug-ins lag die CPU-Last der FAUST-Versionen jeweils unter derjenigen der STK-Versionen; der ausgewiesene Effizienzgewinn betrug je nach Modell zwischen 22,91 Prozent und 80,20 Prozent. Damit stützen die Ergebnisse die Einschätzung, dass FAUST nicht nur eine kompakte Beschreibung von Signalverarbeitungsalgorithmen ermöglicht, sondern auch effizienten und gut dokumentierbaren Code erzeugt. (**michon:fauststk**)

Für die praktische Eignung muss der durch FAUST generierte Code sowohl funktional korrekt, als auch die erforderliche Laufzeiteffizienz erreichen. Scaringella et al. (2003) thematisieren die automatische Vektorisierung, während Letz et al. (2010) Möglichkeiten der Parallelisierung im FAUST-Umfeld behandeln. Die FAUST-Dokumentation verweist zudem auf Optimierungen, die den vollständigen Signalfluss berücksichtigen (Orlarey et al., 2009). Diese Eigenschaften lassen jedoch keine allgemeine Aussage darüber zu, ob FAUST-generierter C++-Code einer manuell implementierten C++-Referenz bei konkreten DSP-Bausteinen überlegen, gleichwertig oder unterlegen ist.

Direkte Vergleiche liegen für zeitbereichsbasierte Algorithmen vor. Für den Reverb-Algorithmus *Freeverb* berichten \parencite{orlarey:hal-02159014}<empty citation> Benchmarkergebnisse, deren Ausprägung von Compiler und Codegenerierungsstrategie abhängt. Die in dieser Arbeit berücksichtigte Vergleichsliteratur untersucht überwiegend zeitbereichsbasierte Algorithmen wie Reverbs, Delays und physikalische Modelle. Ein Vergleich FFT-bezogener Bausteine wird daher ergänzend untersucht. Für einen fairen Vergleich folgen beide Implementierungen demselben algorithmischen Ablauf und werden über eine gemeinsame Schnittstelle in die Benchmark-Infrastruktur eingebunden. Auf explizit programmierte Single-Instruction-Multiple-Data-Intrinsics (SIMD-Intrinsics) wird verzichtet. Die automatische Vektorisierung des Compilers bleibt für beide Varianten aktiv.

Für die vorliegende Arbeit ist insbesondere die Fast-Fourier-Transformation (FFT) relevant. Die FAUST-Dokumentation weist darauf hin, dass das abtastwertorientierte Berechnungsmodell der Sprache bei Multi-Rate-Algorithmen wie FFT und Faltung strukturelle Herausforderungen mit sich bringen kann (Orlarey et al., 2009). Eine Untersuchung einzelner FFT-bezogener DSP-Bausteine ergänzt daher die bisherigen, überwiegend zeitbereichsorientierten Vergleiche.

Die vorliegende Arbeit untersucht systematisch, wie sich FAUST-generierter C++-Code gegenüber einer manuell implementierten C++-Referenz bei elementaren DSP-Bausteinen verhält. Betrachtet werden eine Verstärkungsstufe (Gain-Stufe), ein Biquad-Filter, Finite-Impulse-Response-Filter (FIR-Filter) unterschiedlicher Länge sowie FFTs unterschiedlicher Größe. Der Schwerpunkt liegt auf reproduzierbaren Mikrobenchmarks, nicht auf einer vollständigen Audioanwendung.

1.2 Forschungsfrage und Zielsetzung

Ziel dieser Arbeit ist es, die Eignung der domänenspezifischen Sprache FAUST für die Implementierung ausgewählter Echtzeit-DSP-Bausteine zu untersuchen. Dazu werden FAUST-generierter C++-Code und eine manuell implementierte C++-Referenz und reproduzierbaren Vergleiche mit unterschiedlichen Abhängigkeits- und Skalierungsei-

enschaften erzeugt. Neben der Laufzeitperformanz werden Speicherverbrauch und Binärgröße betrachtet. Ergänzend werden die Codelänge und die Implementierungskomplexität als Aspekte der Entwicklererfahrung analysiert.

Die übergeordnete Forschungsfrage lautet:

Wie unterscheidet sich FAUST-generierter C++-Code von einer manuell implementierten C++-Referenz bei ausgewählten DSP-Bausteinen und FFTs hinsichtlich Laufzeit, Speicherverbrauch, Binärgröße, Codelänge, Implementierungskomplexität und Benutzerfreundlichkeit von FAUST?

Zur Beantwortung dieser Forschungsfrage werden folgende Teilfragen untersucht:

- Wie groß ist der Performanzunterschied hinsichtlich der CPU-Zeit pro Audio-Block zwischen FAUST-generiertem und manuell implementiertem C++-Code bei Gain-Stufen, Biquad-Filtern, FIR-Filtern und FFTs?
- Wie verändert sich der Laufzeitunterschied zwischen beiden Implementierungen in Abhängigkeit von Blockgröße, FIR-Filterlänge und FFT-Größe?
- Wie unterscheiden sich FAUST-generierter und manuell implementierter C++-Code hinsichtlich des Speicherverbrauchs während der Ausführung der untersuchten DSP-Bausteine?
- Welche Unterschiede bestehen zwischen FAUST-generiertem und manuell implementiertem C++-Code hinsichtlich der Binärgröße der erzeugten Programme?
- Welchen Einfluss haben Compileroptimierungen und automatische Vektorisierung auf beide Implementierungen?
- Welche Unterschiede bestehen hinsichtlich Codelänge und Implementierungskomplexität der beiden Ansätze?
- Wie verlief die Einarbeitungsphase und die Umgang mit FAUST in der gesamten Arbeit.

1.3 Aufbau der Arbeit

Die vorliegende Bachelorarbeit ist in acht Kapitel gegliedert. Sie umfasst die theoretischen Grundlagen, die Beschreibung von FAUST, das Versuchsdesign, die Implementierung der Benchmark-Kerne, die systematische Evaluation sowie die Einordnung der Ergebnisse.

Nach dieser Einleitung vermittelt Kapitel 2 die für die Arbeit relevanten Grundlagen der digitalen Signalverarbeitung. Dazu gehören Audio-DSP, Verstärkungsstufen, rekursive- und FIR-Filter, die Fast-Fourier-Transformation sowie Anforderungen an Echtzeitverarbeitung.

Kapitel 3 stellt die domänenspezifische Sprache FAUST vor. Es erläutert die funktionale Syntax und Semantik, die zugrunde liegende Compiler-Architektur sowie die für die Arbeit relevanten Codegenerierungs-Backends.

Kapitel 4 beschreibt das Versuchsdesign und die Benchmark-Methodik. Es definiert die untersuchten DSP-Bausteine, die Messgrößen, die Zielplattformen, die Compilerkonfigurationen und die Maßnahmen zur Sicherung reproduzierbarer Messungen.

Kapitel 5 dokumentiert die praktische Umsetzung der FAUST- und C++-Varianten. Neben den DSP-Bausteinen werden die gemeinsame Schnittstelle, das Build-System und die Maßnahmen zur Gewährleistung einer fairen Vergleichsbasis beschrieben.

Kapitel 6 enthält die Evaluation. Die Ergebnisse der Mikrobenchmarks werden hinsichtlich Laufzeit, Speicherverbrauch und Binärgröße ausgewertet. Zusätzlich werden Skalierungseffekte durch unterschiedliche Blockgrößen, Filterlängen und FFT-Größen sowie der Einfluss von Compileroptimierungen analysiert. Im Anschluss wird die Erfahrung mit der Handhabung von FAUST bewertet.

Kapitel 7 diskutiert die Ergebnisse im Hinblick auf die Forschungsfrage. Dabei werden Limitationen des Versuchsaufbaus, Bedrohungen der Validität und die qualitativen Aspekte der Entwicklererfahrung eingeordnet.

Kapitel 8 fasst die zentralen Ergebnisse zusammen und gibt einen Ausblick auf mögliche Erweiterungen, etwa die Untersuchung vollständiger DSP-Pipelines oder weiterer Zielplattformen.

Kapitel 2

Grundlagen

Dieses Kapitel vermittelt die signaltheoretischen und laufzeitbezogenen Grundlagen, die für den Vergleich von Faust-generiertem und manuell implementiertem C++-Code erforderlich sind. Spezifisch werden folgende Grundlagen der digitale Audiosignale aufgearbeitet, eine Gain-Stufe, Biquad- und FIR-Filter, die Fast-Fourier-Transformation sowie die Anforderungen blockbasierter Echtzeitverarbeitung.

2.1 Digitale Audiosignale und blockweise Verarbeitung

2.1.1 Abtastung und diskrete Sample-Folgen

Ein analoges Audiosignal liegt zeitkontinuierlich vor. Für seine digitale Verarbeitung wird es in regelmäßigen Zeitabständen abgetastet und als diskrete Folge von Sample-Werten dargestellt. Die bei Rossing erläuterte Abtastung ordnet jedem Sample einen festen zeitlichen Abstand zu Rossing, 2007. Die Abtastfrequenz f_s bestimmt die Zahl der Messungen pro Sekunde; das Abtastintervall beträgt

$$T_s = \frac{1}{f_s}. \quad (2.1)$$

Die Beziehung zwischen Abtastfrequenz und Abtastintervall folgt aus der Definition der Abtastfrequenz, die Rossing für die Beschreibung digitaler Audiosignale verwendet (Rossing, 2007, S. 520–522).

Die digitale Darstellung eines kontinuierlichen Signals $x(t)$ lautet damit

$$x[n] = x(nT_s), \quad n \in \mathbb{Z}. \quad (2.2)$$

Rossing beschreibt mit dieser zeitdiskreten Darstellung die Überführung eines kontinuierlichen Signals in eine Sample-Folge (Rossing, 2007, S. 520–522).

Das Nyquist-Shannon-Kriterium fordert, dass die Abtastfrequenz mindestens doppelt so hoch wie die höchste im Signal enthaltene Frequenz sein muss. Andernfalls können Frequenzanteile nicht eindeutig rekonstruiert werden (Rossing, 2007).

$$f_s \geq 2f_{\max}. \quad (2.3)$$

Rossing behandelt diese Bedingung als Grundlage einer aliasfreien digitalen Darstellung (Rossing, 2007, S. 521–522).

In der vorliegenden Arbeit werden Audiosignale als Folgen von Gleitkommawerten verarbeitet. Die Quantisierung ist daher nur als Übergang von der analogen in die digitale Darstellung relevant; eine separate Untersuchung der Bittiefe oder der Quantisierungsqualität ist nicht Gegenstand der Benchmarks. Add citation

2.1.2 Audio-Blöcke und Echtzeitdeadline

Audio-DSP-Systeme übergeben Samples in der Regel blockweise an einen Verarbeitungsalgorithmus. Ein Block enthält B aufeinanderfolgende Samples und besitzt bei der Abtastfrequenz f_s die Dauer

$$T_{\text{Block}} = \frac{B}{f_s}. \quad (2.4)$$

Die Formel ergibt sich aus der Anzahl der Samples pro Block und dem zeitlichen Abstand zwischen zwei Samples. Smith behandelt die blockweise Verarbeitung als praktische Organisationsform digitaler Audiosignale (**smith2007digitalfilters**).

Damit die Verarbeitung echtzeitfähig bleibt, muss die Berechnung eines Blocks vor dem Eintreffen des nächsten Blocks abgeschlossen sein:

$$T_{\text{Berechnung}} < T_{\text{Block}}. \quad (2.5)$$

Diese Ungleichung definiert die Echtzeitdeadline für eine blockweise Verarbeitung und folgt direkt aus der verfügbaren Blockdauer. Eine kleinere Blockgröße reduziert die durch die Pufferung verursachte Latenz, verkürzt aber zugleich das verfügbare Zeitfenster für die Berechnung. Die Blockgröße ist deshalb ein zentraler Parameter der Benchmark-Konfiguration.

2.2 Elementare DSP-Bausteine

2.2.1 Gain-Stufe

Eine Gain-Stufe skaliert jedes Eingangssample mit einem konstanten Verstärkungsfaktor g . Für ein Eingangssignal $x[n]$ ergibt sich das Ausgangssignal $y[n]$ zu

$$y[n] = g \cdot x[n]. \quad (2.6)$$

Smith behandelt diese Skalierung als elementare lineare Operation der digitalen Audioverarbeitung (**smith2007digitalfilters**).

Der Baustein benötigt pro Sample lediglich eine Multiplikation und dient deshalb als einfacher Referenzfall für den Vergleich des durch Faust erzeugten Codes mit der C++-Referenz. Bei einer Angabe der Verstärkung in Dezibel gilt

$$g = 10^{G_{\text{dB}}/20}. \quad (2.7)$$

Smith leitet diese Umrechnung aus der Definition eines Amplitudenverhältnisses in Dezibel ab (**smith2007digitalfilters**).

Die folgende Abbildung veranschaulicht eine Gain-Stufe mit $g = 0.5$. Das Ausgangssignal besitzt für jedes Sample die halbe Amplitude des Eingangssignals, was einer Absenkung von ungefähr -6.02 dB entspricht.

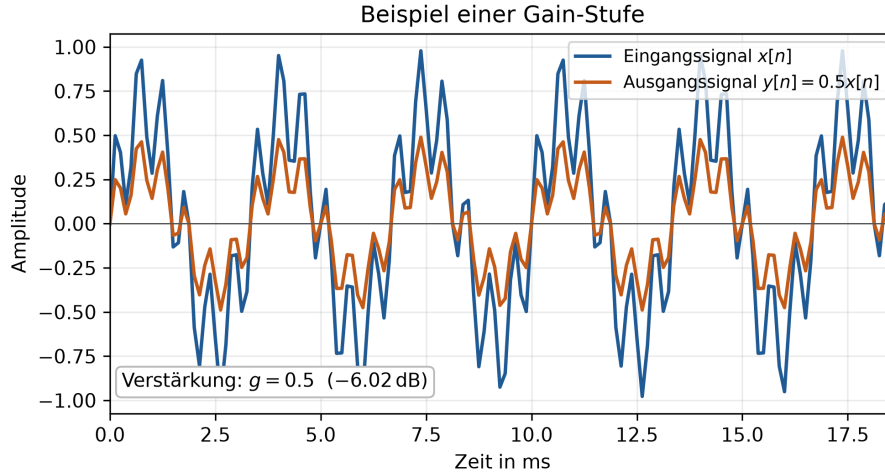


Abbildung 2.1: Beispiel einer Gain-Stufe mit $g = 0.5$. Die Amplitude jedes Eingangssamples wird halbiert. Eigene Darstellung nach (smith2007digitalfilters).

Die Gain-Stufe bildet damit einen bewusst einfachen, vollständig datenparallelen Benchmark-Kern. Scaringella et al. beschreiben Vektorisierung als Ausführung derselben Operation auf mehreren Datenwerten durch SIMD-Instruktionen; eine sampleweise Skalierung besitzt daher grundsätzlich ein hohes Vektorisierungspotenzial (Scaringella et al., 2003).

2.2.2 IIR-Filter und Biquad-Struktur

Ein Infinite-Impulse-Response-Filter (IIR-Filter) verwendet neben aktuellen und vergangenen Eingangswerten auch vergangene Ausgangswerte. Die Rückkopplung führt dazu, dass der Filterzustand über mehrere Samples erhalten bleibt. Für die Untersuchung wird ein Biquad-Filter, also ein IIR-Filter zweiter Ordnung, verwendet. Seine allgemeine Differenzengleichung lautet bei normiertem $a_0 = 1$

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] - a_1y[n-1] - a_2y[n-2]. \quad (2.8)$$

Smith leitet IIR-Filter über solche Differenzengleichungen her. Die Koeffizienten b_i beschreiben den Vorwärtzweig, während die Koeffizienten a_i den Rückkopplungsweig bestimmen (smith2007digitalfilters).

Die folgende Abbildung zeigt die Wirkung eines Biquad-IIR-Tiefpasses zweiter Ordnung mit einer Grenzfrequenz von 800 Hz. Wie beim FIR-Beispiel enthält das synthetische Eingangssignal Anteile bei 300 Hz und 1800 Hz. Der Frequenzgang und das Ausgangssignal veranschaulichen die Dämpfung hoher Frequenzanteile durch die rekursive Filterstruktur.

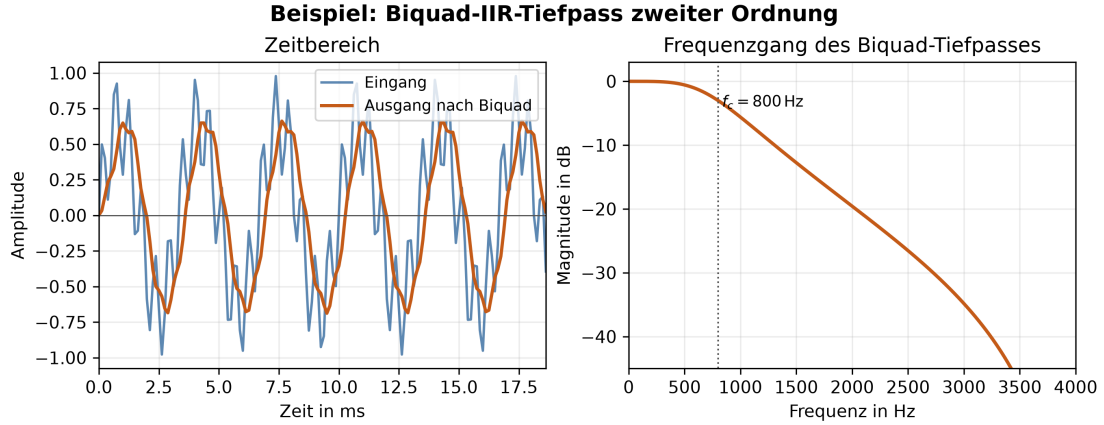


Abbildung 2.2: Beispiel eines Biquad-IIR-Tiefpasses zweiter Ordnung mit einer Grenzfrequenz von $f_c = 800$ Hz. Der Frequenzgang zeigt die Dämpfung hoher Frequenzanteile. Eigene Darstellung nach (**smith2007digitalfilters**).

Eine IIR-Implementierung muss die vergangenen Ein- und Ausgangswerte als Zustände speichern und bei jedem Sample aktualisieren. Für den Benchmark ist besonders relevant, dass diese Rückkopplung Datenabhängigkeiten erzeugt und die Vektorisierbarkeit gegenüber rein vorwärtsgerichteten Operationen einschränken kann. Scaringella et al. unterscheiden entsprechend zwischen vektorisierbaren und skalaren Ausdrücken und behandeln rekursive Filter als wichtigen Sonderfall (Scaringella et al., 2003).

2.2.3 FIR-Filter und Faltung

Ein Finite-Impulse-Response-Filter (FIR-Filter) besitzt keine Rückkopplung. Jedes Ausgangssample entsteht aus einer gewichteten Summe aktueller und vergangener Eingangssamples. Für eine Filterlänge L gilt

$$y[n] = \sum_{k=0}^{L-1} b_k x[n - k]. \quad (2.9)$$

Smith zeigt, dass diese Differenzengleichung der direkten Faltungsimplementierung eines FIR-Filters entspricht (**smith2007digitalfilters**).

Die folgende Abbildung zeigt einen FIR-Tiefpass mit 31 Taps und einer Grenzfrequenz von 800 Hz. Das synthetische Eingangssignal enthält einen niederfrequenten Anteil bei 300 Hz sowie einen höherfrequenten Anteil bei 1800 Hz; der FIR-Filter dämpft den höherfrequenten Anteil.

Die Koeffizienten b_k werden als Taps bezeichnet. Im Unterschied zum Biquad hängt $y[n]$ nicht von früheren Ausgangswerten ab. Der Aufwand wächst mit der Zahl der Taps. Pro Ausgangssample beträgt die asymptotische Komplexität daher

$$O(L). \quad (2.10)$$

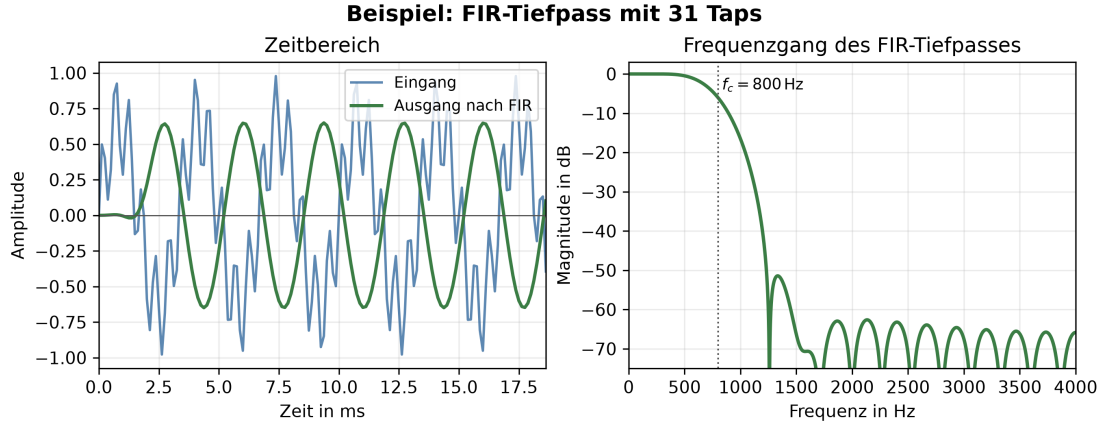


Abbildung 2.3: Beispiel eines FIR-Tiefpasses mit 31 Taps und einer Grenzfrequenz von $f_c = 800$ Hz. Der Filter arbeitet ohne Rückkopplung und dämpft hohe Frequenzanteile. Eigene Darstellung nach (**smith2007digitalfilters**).

Die lineare Abhängigkeit von der Filterlänge ergibt sich aus der direkten Auswertung der Faltungssumme, wie Smith ausführt (**smith2007digitalfilters**).

Die Filterlänge ist für die Evaluation wesentlich: Sie erhöht sowohl die Zahl der Operationen als auch die Größe des benötigten Zustands. Scaringella et al. weisen darauf hin, dass nichtrekursive Filterstrukturen vollständig vektorisiert werden können, sofern keine weiteren Datenabhängigkeiten entgegenstehen (Scaringella et al., 2003).

2.3 Frequenzanalyse mit DFT und FFT

2.3.1 Zeitbereich, Frequenzbereich und DFT

Im Zeitbereich beschreibt ein digitales Signal seinen Amplitudenverlauf über dem Sample-Index. Im Frequenzbereich wird derselbe Signalabschnitt durch seine spektralen Komponenten dargestellt. Tan und Jiang ordnen mit der diskreten Fourier-Transformation (DFT) einer endlichen Folge $x[n]$ der Länge N komplexe Koeffizienten $X[k]$ zu:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}, \quad k = 0, \dots, N-1. \quad (2.11)$$

Tan und Jiang erläutern die DFT-Definition sowie die Bedeutung der Indizes n , k und N (Tan & Jiang, 2018, S. 97).

Der Betrag eines Koeffizienten beschreibt die Stärke der jeweiligen Frequenzkomponente, während sein Argument die zugehörige Phase enthält. Der Abstand zwischen benachbarten Frequenz-Bins beträgt

$$\Delta f = \frac{f_s}{N}. \quad (2.12)$$

In der folgenden Abbildung

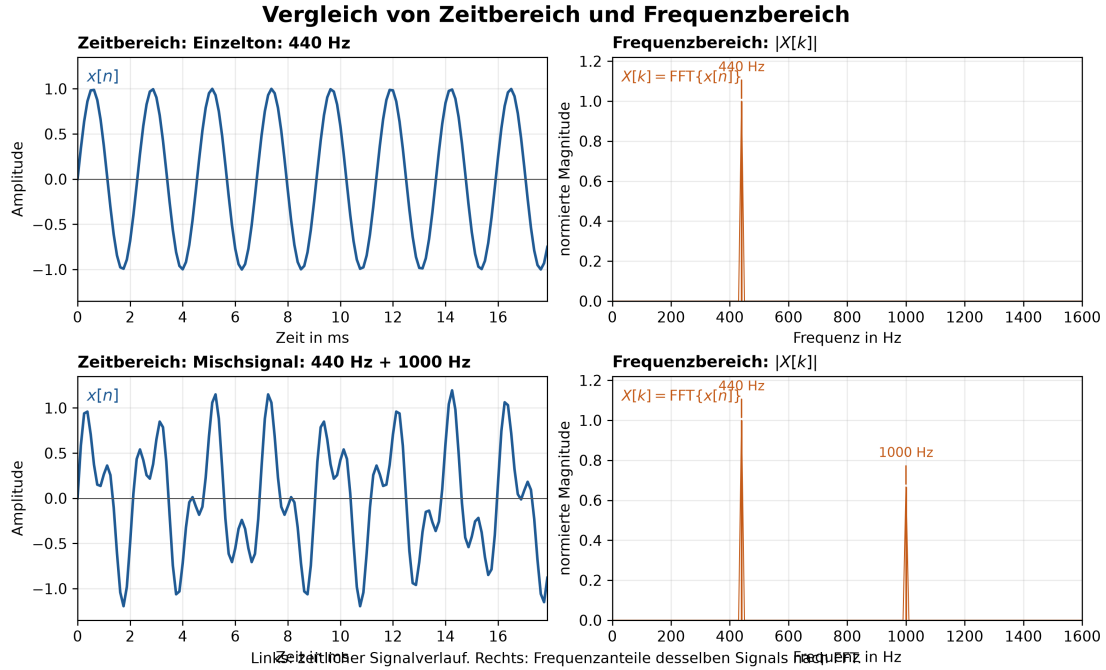


Abbildung 2.4: Vergleich von Zeitbereich und Frequenzbereich. Oben ist ein 440-Hz-Einzelton dargestellt, der im Spektrum einen dominanten Frequenzanteil bei 440 Hz erzeugt. Unten ist ein Mischsignal aus 440 Hz und 1000 Hz dargestellt; beide Anteile erscheinen im Frequenzbereich als separate Peaks. Eigene Darstellung auf Grundlage der DFT/FFT-Spektralanalyse (Tan & Jiang, 2018, S. 102).

Tan und Jiang leiten diese Frequenzauflösung aus Abtastfrequenz und Transformationslänge ab (Tan & Jiang, 2018, S. 101, 103). Eine größere Transformationslänge N verbessert folglich die Frequenzauflösung, erhöht jedoch den Rechenaufwand.

2.3.2 Fast-Fourier-Transformation

Die Fast-Fourier-Transformation (FFT) berechnet dieselben DFT-Koeffizienten mit einer effizienteren Zerlegung des Problems. Während eine direkte DFT eine Komplexität von

$$O(N^2) \quad (2.13)$$

aufweist, reduziert eine radix-2-FFT den Aufwand auf

$$O(N \log_2 N). \quad (2.14)$$

Tan und Jiang erklären diese Beschleunigung durch die Rekursion der N -Punkt-DFT in kleinere Teiltransformationen. Die asymptotischen Laufzeiten werden dort als Konsequenz dieser Zerlegung begründet (Tan & Jiang, 2018, S. 127–134).

Die FFT-Größe stellt deshalb im Benchmark einen relevanten Skalierungsparameter dar: Sie verändert sowohl den theoretischen Rechenaufwand als auch die praktische

Speicher- und Cache-Nutzung. Dieses Kapitel behandelt ausschließlich die Vorwärtstransformation, weil die Forschungsfrage einzelne FFT-Bausteine vergleicht; eine vollständige STFT- oder Overlap-Add-Pipeline ist nicht Teil dieses Grundlagenkapitels.

2.4 Performanzrelevante Implementierungsaspekte

2.4.1 Compileroptimierung und automatische Vektorisierung

Die Laufzeit eines DSP-Kerns wird nicht allein durch seinen Algorithmus bestimmt, sondern auch durch die Codegenerierung und die Optimierungen des Compilers. Bei automatischer Vektorisierung versucht der Compiler, mehrere unabhängige skalare Operationen durch SIMD-Instruktionen zusammenzufassen. Scaringella et al. beschreiben für Faust eine Analyse, die vektorisierbare Ausdrücke von Ausdrücken mit Datenabhängigkeiten trennt. Diese Unterscheidung erklärt, warum Gain- und lange FIR-Strukturen andere Optimierungsmöglichkeiten besitzen können als rekursive Biquad-Filter (Scaringella et al., 2003).

Die Arbeit vergleicht beide Implementierungen unter identischen Compiler- und Optimierungsbedingungen. Explizite SIMD-Intrinsics werden nicht eingesetzt, damit die Wirkung der automatischen Optimierung für Faust und C++ beobachtbar bleibt. Die konkrete Wahl von Compiler, Flags und Zielplattform wird erst im Versuchsdesign dokumentiert.

2.4.2 Messgrößen

Die zentrale Messgröße ist die CPU-Zeit pro Audio-Block. Ergänzend werden Speicherverbrauch und Binärgröße erfasst, weil sie die praktische Einsetzbarkeit einer Implementierung neben der reinen Laufzeit beeinflussen. Codelänge und Implementierungskomplexität ergänzen den quantitativen Vergleich um Aspekte der Entwicklererfahrung. Die genaue Operationalisierung dieser Größen, die Zahl der Wiederholungen sowie die statistische Auswertung gehören in Kapitel 4.

2.5 Zusammenfassung

Die Grundlagen beschreiben die für die Benchmarks benötigte Verarbeitungskette von digitalen Samples über Gain-, IIR- und FIR-Operationen bis zur FFT. Blockgröße, Filterlänge und FFT-Größe bestimmen dabei den Rechenaufwand und bilden die zentralen Skalierungsparameter der Evaluation. Die folgenden Kapitel erläutern anschließend Faust und das konkrete Versuchsdesign, ohne die Grundlagen um anwendungsfremde Verfahren der Sprachrauschunterdrückung zu erweitern.

Kapitel 3

Faust: Functional Audio Stream

3.1 Einführung

FAUST ist eine Programmiersprache für Audio-Signalverarbeitung. Sie wurde entwickelt, um eine Alternative zu C/C++ zu bieten, mit der sich effizient Signalverarbeitungsbibliotheken, Audio-Plug-ins und eigenständige Anwendungen erstellen lassen. Vgl. Orlarey et al., 2009, S. 1

Anders als visuelle Sprachen wie Max oder Pure Data ist FAUST eine kompilierte Sprache. Der FAUST-Compiler übersetzt FAUST-Programme direkt in C++-Code. Dieser Code ist lauffähig, selbstständig und benötigt keine externe Laufzeitumgebung. Er arbeitet deterministisch und mit konstantem Speicherverbrauch. Vgl. Orlarey et al., 2009, S. 2

Die Besonderheit von FAUST liegt in der formalen Semantik: Jedes Programm beschreibt eine mathematische Funktion, die Eingangssignale in Ausgangssignale transformiert. Der Compiler übersetzt nicht den Programmtext, sondern die dahinterstehende Funktion. Dadurch können zwei syntaktisch unterschiedliche, aber semantisch äquivalente Programme denselben Code erzeugen. Vgl. Orlarey et al., 2009, S. 2

3.2 Programmmodell

FAUST kombiniert funktionale Programmierung mit Blockdiagrammen. Digitale Signale sind diskrete Funktionen der Zeit, Signalprozessoren sind Funktionen auf diesen Signalen, und Kompositionsoperatoren setzen mehrere Signalprozessoren zu größeren Blockdiagrammen zusammen. Ein FAUST-Programm beschreibt keinen Klang, sondern einen Signalprozessor, also eine Maschine mit Eingängen und Ausgängen. Das Schlüsselwort `process` definiert diesen Signalprozessor, vergleichbar mit `main` in C. Vgl. Orlarey et al., 2009, S. 3

```
1 process = 0;           // erzeugt Stille
2 process = _;           // kopiert Eingang auf Ausgang
3 process = +;           // addiert zwei Kanäle (Stereo zu Mono)
```

FAUST-Primitive funktionieren wie C-Funktionen, aber auf Signalen: `sin` wendet die Sinus-Funktion sampleweise an, `@` ist ein Delay-Operator mit $Y(t) = X(t - D(t))$. Vgl. Orlarey et al., 2009, S. 4

3.3 Syntax: Blockdiagramm-Komposition

FAUST stellt fünf Kompositionsoperatoren bereit, die als Verallgemeinerung der Funktionskomposition $\circ$ zu verstehen sind: Vgl. Orlarey et al., 2009, S. 4

Tabelle 3.1: Kompositionsoperatoren der Faust Block-Diagramm-Algebra

Operator	Bezeichnung	Beschreibung
$f \sim g$	Rekursive Komposition	Feedback-Schleife mit 1-Sample-Delay
f, g	Parallele Komposition	Zwei Signalprozessoren nebeneinander
$f : g$	Sequenzielle Komposition	Ausgang von f mit Eingang von g verbinden
$f < : g$	Split	Ein Ausgang auf mehrere Eingänge verteilen
$f > : g$	Merge	Mehrere Ausgänge auf einen Eingang zusammenführen

Beispiel für sequenzielle Komposition: `process = + : abs;` verbindet die Ausgabe einer Addition mit einem Absolutwert. Vgl. Orlarey et al., 2009, S. 4

Beispiel für rekursive Komposition: `process = + _;` erzeugt einen Integrator mit $Y(t) = X(t) + Y(t - 1)$. Vgl. Orlarey et al., 2009, S. 4

Ein vollständigeres Beispiel ist ein Rauschgenerator: Vgl. Orlarey et al., 2009, S. 5

```
1  random = +(12345) ~ *(1103515245);
2  noise = random / 2147483647.0;
3  process = noise * vslider("noise[style:knob]", 0, 0, 100, 0.1) / 100;
```

Vgl. Orlarey et al., 2009, S. 5

Die erste Zeile nutzt eine rekursive Komposition mit einem 1-Sample-Delay, um eine Pseudo-Zufallszahl gemäß der Formel $R(t) = 12345 + 1103515245 \cdot R(t - 1)$ zu berechnen. Die weiteren Zeilen skalieren diesen Wert in den Bereich $[-1, 1]$ und fügen einen interaktiven Lautstärkeregler hinzu, um das resultierende Signal zu steuern.

3.4 Semantik

FAUST basiert auf einer denotationale Semantik, sie beschreibt die Bedeutung eines Programms, indem sie es direkt als mathematisches Objekt definiert, in diesem Fall als eine Funktion f , die Eingangssignale in Ausgangssignale umwandelt. Anstatt den Ablauf der Berechnung Schritt für Schritt zu beschreiben, wird das Programm als ein fertiger Signalprozessor betrachtet, der eine mathematische Abbildung zwischen Signalfolgen darstellt. Dies ermöglicht es, komplexe Systeme durch die Kombination solcher Funktionen mathematisch präzise zu beschreiben.

3.5 Compiler-Architektur

Der FAUST-Compiler arbeitet in mehreren Schritten, bevor C++-Code erzeugt wird. Vgl. Orlarey et al., 2009, S. 10-12

1. **Parsen.** Alle Quelldateien werden eingelesen und in eine Liste von Definitionen umgewandelt. Redefinitionen sind nicht erlaubt.

2. **Blockdiagramm-Auswertung.** Algorithmische Definitionen werden in flache, primitive Form gebracht (Normalform).
3. **Semantik entdecken.** Symbolische Signale werden durch das Blockdiagramm propagiert, um die mathematische Gleichung jedes Ausgangssignals zu erhalten. Verschiedene Blockdiagramme, die dasselbe berechnen, ergeben die gleichen Gleichungen.

Zwei äquivalente Programme:

```

1   process = /(2) : @ (10);
2   process = *(2) : @ (7) : /(4) : @ (3);
3
```

Beide ergeben $Y(t) = 0.5 \cdot X(t-10)$. Der Compiler normalisiert solche Gleichungen, um Delay-Lines zu teilen und Operationen zu vereinfachen.

4. **Typzuweisung** Jedes Signal bekommt einen Typ mit fünf Eigenschaften:
 - **Datentyp** Integer oder Float
 - **Wertebereich** Minimale und maximale Werte eines Signals
 - **Berechnungszeitpunkt** Zur Kompilier-, Initialisierungs- oder Laufzeit
 - **Geschwindigkeit** Konstant, langsam (pro Block) oder schnell (pro Sample)
 - **Parallelisierbarkeit** Ja bei unabhängigen Signalen, aber nicht bei rekursiven Abhängigkeiten

Aus Benutzersicht sind alle Signale volle Audiosignale; die Geschwindigkeitsunterscheidung dient nur der internen Optimierung.

5. **Vorkommenanalyse** Teilausdrücke werden auf Wiederverwendbarkeit geprüft. Häufig verwendete Ausdrücke werden in Cache-Variablen ausgelagert.

3.6 Codegenerierung

FAUST bietet drei Codegenerierungsstrategien, die aufeinander aufbauen: Vgl. Orlarey et al., 2009, S. 12-20

3.6.1 Skalarer Modus

Ein einzelner Loop berechnet sampleweise die Ausgänge. Für jede Addition $E_1 + E_2$ wird der Code von E_1 und E_2 generiert und mit einem $+$ verbunden. Wird das Ergebnis mehrfach verwendet, wird eine Zwischenspeicher-Variable angelegt.

```

1  // Stereo zu Mono: kein Caching nötig
2  process = +;
3  $\rightarrow$ output0[i] = input0[i] + input1[i];
4
5  // Split: Ergebnis wird zwischengespeichert
6  process = + <: _,_;
7  $\rightarrow$ float fTemp0 = input0[i] + input1[i];
8  output0[i] = fTemp0;
9  output1[i] = fTemp0;
```

Vgl. Orlarey et al., 2009, S. 13

3.6.2 Vektor-Modus

Moderne C++-Compiler nutzen SIMD-Instruktionen für Vektoroperationen (theoretischer Speedup: 4x). Damit diese Vektorisierung funktioniert, darf der Code nicht zu komplex verschachtelt sein.

Der Vektor-Modus teilt den einen großen Loop in mehrere kleine, einfache Loops auf, die über Vektoren kommunizieren. Komplexe oder gemeinsam benutzte Ausdrücke bekommen eigene Loops. Das Ergebnis ist ein gerichteter Graph (DAG) aus Schleifen.

3.6.3 Parallel-Modus

Der Parallel-Modus fügt OpenMP-Direktiven in den Vektor-Code ein, um unabhängige Schleifen parallel auf mehreren CPU-Kernen auszuführen. Der Algorithmus: Der topologisch sortierte Schleifengraph wird in Ebenen eingeteilt. Schleifen derselben Ebene sind unabhängig und können parallel laufen. Für jede Ebene erzeugt der Compiler eine `#pragma ompsections`-Direktive. Enthält eine Ebene nur eine Schleife, wird geprüft, ob diese parallelisierbar ist (keine rekursiven Abhängigkeiten). Vgl. Orlarey et al., 2009, S. 16-21

3.6.4 Generierte Code

Der erzeugte C++-Code ist eine Klasse mit diesen Kernmethoden:

- `getNumInputs()` / `getNumOutputs()` - Anzahl Ein-/Ausgänge
- `init(int samplingFreq)` - Initialisierung
- `buildUserInterface(UI*)` - GUI-Beschreibung
- `compute(int count, float** input, float** output)` - Signalverarbeitungsmethode

3.7 Backends und Architekturdateien

FAUST trennt die Signalverarbeitungslogik von der Einbettung in Audio-Systeme. Dazu dienen Architekturdateien, die als Wrapper zwischen dem erzeugten C++-Code und dem Zielsystem stehen. Ein einziges FAUST-Programm kann so ohne Änderung für alle diese Formate Vgl. Orlarey et al., 2009, Tabelle 3, S. 7 kompiliert werden. Neue Architekturdateien lassen sich einfach hinzufügen. Vgl. Orlarey et al., 2009, S. 7

3.8 Performance

Benchmarks mit drei Testprogrammen auf verschiedenen Hardware-Konfigurationen zeigen: Vgl. Orlarey et al., 2009, S. 22-28

- **Einfache Programme** (z. B. Signal kopieren): Der skalare Modus ist dem Vektor-Modus ebenbürtig. Parallele Versionen sind langsamer, da die Speicherbandbreite bereits ausgelastet ist. Vgl. Orlarey et al., 2009, S. 22
- **Programme mit mittlerem Parallelismus** (z. B. Reverb): ICC zeigt moderate Speedups bei Vektorisierung und Parallelisierung. Vgl. Orlarey et al., 2009, S. 22-23
- **Programme mit hohem Parallelismus** (z. B. 32 parallele Strings): Deutliche Performance-Steigerung vom skalaren über den vektorisierten zum parallelen Modus. Vgl. Orlarey et al., 2009, S. 23

Die Wirksamkeit hängt also stark vom jeweiligen Programm ab.

3.9 Zusammenfassung

FAUST ist eine kompilierte Sprache für Audio-Signalverarbeitung mit klarer formaler Semantik. Der Compiler erzeugt effizienten, eigenständigen C++-Code, der mit handgeschriebenem Code konkurrieren kann. Die drei Codegenerierungsstrategien (skalar, vektorisiert, parallel) und die Vielzahl von Architekturdateien machen FAUST flexibel für unterschiedliche Einsatzzwecke.

Kapitel 4

Mathematical Elements, Equations and Algorithms

Kapitel 5

Using Literature and other Resources

Kapitel 6

Printing the Manuscript

Kapitel 7

Closing Remarks

Anhang A

Technical Details

Anhang B

Supplementary Materials

List of supplementary data submitted to the degree-granting institution for archival storage (in ZIP format).

B.1 PDF Files

Pfad: /

thesis.pdf Master/Bachelor thesis (complete document)

B.2 Media Files

Pfad: /media

*.ai, *.pdf Adobe Illustrator files
*.jpg, *.png raster images
*.mp3 audio files
*.mp4 video files

Anhang C

Questionnaire

Anhang D

LaTeX Source Code

Quellenverzeichnis

Literatur

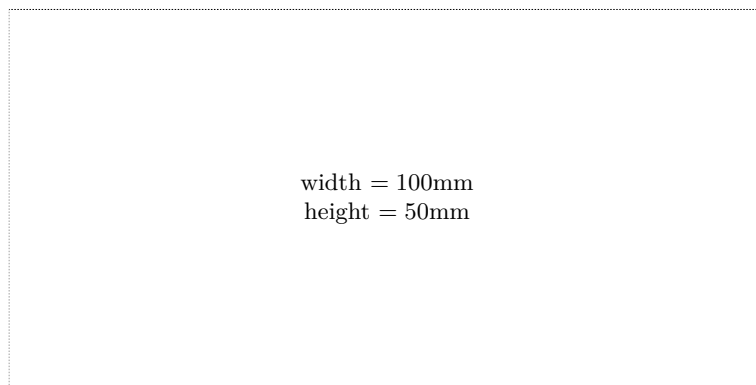
- Ifeachor, E. C., & Jervis, B. W. (2002). *Digital signal processing: a practical approach*. Pearson Education.
- Letz, S., Orlarey, Y., & Foer, D. (2010). Work Stealing Scheduler for Automatic Parallelization in Faust. In LAC (Hrsg.), *Proceedings of Linux Audio Conference*. <https://hal.science/hal-02158924> (siehe S. 2).
- Levine, N., & Skolnick, D. (1997). DSP 101 Part 3: Implement Algorithms on a Hardware Platform. *Analog Dialogue*, 31(3). Verfügbar 4. August 2026 unter <https://www.analog.com/en/resources/analog-dialogue/articles/dsp-101-part-3.html>
- Orlarey, Y., Foer, D., & Letz, S. (2009). FAUST : an Efficient Functional Approach to DSP Programming. In E. D. FRANCE (Hrsg.), *NEW COMPUTATIONAL PARADIGMS FOR COMPUTER MUSIC* (S. 65–96). <https://hal.science/hal-02159014> (siehe S. 1, 2, 12–16).
- Rossing, T. (2007). *Springer Handbook of Acoustics*. Springer New York. https://books.google.at/books?id=4ktVwGe_dSMC (siehe S. 5, 6).
- Scaringella, N., Orlarey, Y., Letz, S., & Foer, D. (2003). Automatic vectorization in Faust. In JIM (Hrsg.), *Actes des Journées d'Informatique Musicale JIM2003*. <https://hal.science/hal-02158949> (siehe S. 2, 7–9, 11).
- Tan, L., & Jiang, J. (2018). *Digital Signal Processing: Fundamentals and Applications*. Academic Press. <https://books.google.at/books?id=MxlxDwAAQBAJ> (siehe S. 9, 10).

Online-Quellen

- GRAME - Centre National de Création Musicale. (2025). *Faust Manual*. Verfügbar 20. Oktober 2025 unter <https://faustdoc.grame.fr/manual/introduction/> (siehe S. 1).

Check Final Print Size

— Check final print size! —



— Remove this page after printing! —